Infosys Springboard Virtual Internship

Project Name: AI-Based Employee Wellness Management Platform

Milestone 1 - Employee Feedback Ingestion, Preprocessing & Sentiment Analysis

Date: 16 October 2026

Introduction

The AI-Based Employee Wellness Management Platform is a web application for understanding employee feedback and identifying workplace sentiment. Milestone 1 establishes the complete foundation for later wellness features: authenticated access, validated feedback ingestion, natural-language preprocessing, VADER sentiment classification, structured reports, and a Streamlit interface.

The implemented user journey is:

Landing Page -> Login/Registration -> Connect Database -> Input Feedback -> Sentiment Analysis

Key Highlights

- Built a Streamlit application with landing, registration, login, database connection, input, and sentiment-analysis stages.
- Added local SQLite persistence for user accounts with salted PBKDF2 password hashes.
- Supported direct text, UTF-8 TXT files, and CSV files with configurable text columns.
- Implemented the documented preprocessing order: cleaning, tokenization, stop-word filtering, and lemmatization.
- Integrated NLTK and VADER for preprocessing and sentiment scoring.
- Produced positive, negative, and neutral labels with numeric negative, neutral, positive, and compound scores.
- Added human-readable and JSON reports, CLI access, tests, and Docker/Render deployment configuration.

Technology Stack

- **Frontend:** Streamlit
- **Backend:** Python 3.10+
- **Natural Language Processing:** NLTK WordNet lemmatization and stop-word corpus
- **Sentiment Analysis:** VADER SentimentIntensityAnalyzer
- **Data Storage:** SQLite for local authentication persistence
- **Input Formats:** Direct text, UTF-8 `.txt`, and `.csv`
- **Packaging:** `pyproject.toml` with editable installation and CLI entry point
- **Deployment:** Dockerfile, Streamlit health check, and Render Blueprint
- **Testing:** pytest
- **Code Quality:** Ruff

Project Structure

```
AI_Based_Employee_Wellness_Management_Platform/
■■■ app.py                          # Streamlit frontend and user flow
■■■ pyproject.toml                  # Package metadata and dependencies
■■■ Dockerfile                      # Container deployment definition
■■■ render.yaml                     # Render Blueprint configuration
■■■ wellness/
■   ■■■ cli.py                      # Command-line entry point
■   ■■■ errors.py                   # Explicit domain errors
■   ■■■ ingestion.py                # TXT/CSV validation and reading
■   ■■■ pipeline.py                 # Public text and file analysis API
■   ■■■ preprocessing.py            # NLTK preprocessing pipeline
■   ■■■ report.py                   # Typed reports and serialization
■   ■■■ storage.py                  # SQLite setup and authentication
■■■ tests/
■   ■■■ test_pipeline.py            # Input, preprocessing, sentiment, and UI adapter tests
■   ■■■ test_storage.py             # Database and authentication tests
■■■ docs/
    ■■■ Milestone1_Documentation.md
```

Objectives Achieved

1. Authenticated Application Flow

The landing page routes users to login or registration. Registration validates name, email, password length, and password confirmation. Login checks the stored PBKDF2 password hash. Authenticated users must explicitly connect the SQLite database before submitting feedback.

2. Flexible Feedback Input

The application accepts direct text through a text area and file uploads for TXT and CSV formats. TXT input is decoded as UTF-8. CSV input requires a header and a

non-empty text column. The columns `text`, `feedback`, `comment`, `message`, and `content` are detected automatically, while `--text-column` or the UI field can select a custom column.

3. Ordered Text Preprocessing

The backend applies the required sequence:

1. **Cleaning:** lowercases text, removes URLs, punctuation, numbers, and excess whitespace.
2. **Tokenization:** splits the cleaned text into word tokens with NLTK.
3. **Stop-word filtering:** removes English stop words and non-alphabetic tokens.
4. **Lemmatization:** uses WordNet lemmatization with NLTK part-of-speech tags.

4. Sentiment Analysis and Reporting

VADER calculates `neg`, `neu`, `pos`, and `compound` scores. Classification uses standard compound thresholds:

- `compound >= 0.05`: positive
- `compound <= -0.05`: negative
- otherwise: neutral

Each analysis returns an `AnalysisReport` containing source metadata, original text, cleaned text, processed text, final tokens, scores, label, and counts. The report can be displayed as readable text or serialized to JSON.

5. Validation and Error Handling

The application rejects missing paths, unsupported extensions, invalid UTF-8, missing CSV headers, missing text columns, empty files, blank direct text, missing NLTK data, invalid registration details, duplicate emails, and incorrect login credentials with explicit user-facing errors.

Module Implementation

Module 1: Ingestion (`wellness/ingestion.py`)

This module validates direct text and reads supported files. It keeps CSV-specific logic separate from sentiment processing and combines all non-empty values in the selected text column for analysis.

Module 2: Preprocessing (`wellness/preprocessing.py`)

This module validates required NLTK resources and performs cleaning, tokenization, stop-word filtering, POS tagging, and lemmatization. Missing resources produce a setup message instead of a silent fallback.

Module 3: Sentiment Pipeline (`wellness/pipeline.py`)

`analyze_text()` is the reusable public API for direct feedback. `analyze_file()` uses the ingestion layer and then delegates to the same text pipeline, ensuring direct input and uploads behave consistently.

Module 4: Reports (`wellness/report.py`)

Typed dataclasses model input summaries, scores, labels, tokens, counts, and the complete analysis result. `to_dict()` provides machine-readable output and `to_human_readable()` provides a user-facing summary.

Module 5: Authentication and Database (`wellness/storage.py`)

SQLite creates the `users` table on connection. Passwords are never stored in plain text; each password receives a random salt and PBKDF2-HMAC-SHA256 derivation. Parameterized SQL is used for account creation and lookup.

Module 6: Streamlit Frontend (`app.py`)

The UI owns navigation, forms, upload handling, session state, and presentation. It delegates all validation, ingestion, preprocessing, sentiment analysis, and authentication to the backend modules.

System Flow

1. The user opens the Streamlit landing page.
2. The user registers or logs in.
3. The application initializes and verifies the SQLite database.
4. The user enters direct feedback or uploads TXT/CSV data.
5. The ingestion layer validates and reads the input.
6. The preprocessing layer cleans, tokenizes, filters, and lemmatizes the text.
7. VADER calculates sentiment scores and the pipeline assigns a label.

8. The UI renders the label, numeric scores, processed report, and JSON download.

How to Run

Local Python Setup

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
python -m pip install -e ".[dev,ui]"
python -m wellness.cli --download-nltk-data
streamlit run app.py
```

Open `http://localhost:8501`.

CLI Examples

```
wellness-analyze --text "I feel supported and motivated at work."
wellness-analyze --text "The workload is exhausting." --json
wellness-analyze --file feedback.txt
wellness-analyze --file feedback.csv --text-column feedback --json
```

Docker

```
docker build -t employee-wellness-platform .
docker run --rm -p 8501:8501 employee-wellness-platform
```

The image installs NLTK datasets during build and exposes `/_stcore/health` for container health checks. `render.yaml` can be used to create a Render Blueprint deployment.

Testing and Verification

The completed verification included:

```
13 tests passed
Ruff lint: All checks passed
Streamlit health endpoint: HTTP 200
Streamlit application page: HTTP 200
```

Tests cover representative sentiment, preprocessing behavior, direct text, TXT, CSV, invalid and empty inputs, Streamlit upload delegation, database initialization, registration, password hashing, and authentication.

Known Limitations

- SQLite is local file storage and is not suitable for multi-instance production deployments without a persistent volume or managed database.
- NLTK datasets must be downloaded during local setup; the Dockerfile installs them

automatically.

• CSV analysis combines all non-empty values in the selected text column into one report rather than producing one report per row.

• The UI currently provides a single authenticated user experience without separate employee, manager, or administrator roles.

• VADER is lexicon-based and may not capture domain-specific wellness language, sarcasm, or multilingual feedback accurately.

Future Scope

• Replace local SQLite with a managed persistent database.

• Add role-based access and organization-level data isolation.

• Store analysis history and trend sentiment over time.

• Add multilingual preprocessing and domain-specific sentiment calibration.

• Introduce wellness recommendations only after the core analytics foundation is validated.

• Add dashboards and privacy controls for aggregated employee insights.

Conclusion

Milestone 1 establishes a complete and maintainable foundation for the AI-Based Employee Wellness Management Platform. It combines a clear Streamlit workflow with reusable backend services for validated ingestion, NLTK preprocessing, VADER sentiment analysis, structured reporting, and SQLite authentication. The test suite, lint checks, local health verification, Docker configuration, and Render Blueprint provide a practical path from local evaluation to hosted deployment while keeping future wellness features separate from the current milestone.